

Exoplanet Transit Detection Pipeline

Technical Documentation

IUBAT · CSC 488 · June 2026

Covers:

- Section 1 — Prototype (HTML) Usage Guide
- Section 2 — Colab Notebook Guide
- Section 3 — API and Data Sources
- Section 4 — ML Models and Metrics Reference

Section 1 — Prototype Usage Guide

The `exoplanet_prototype.html` file is a fully self-contained browser application for exploring exoplanet transit detection results. It requires no installation, no Python, and no backend server.

1.1 Opening the Prototype

1. Locate `exoplanet_prototype.html` in your project folder.
2. Double-click the file — it will open in your default web browser.
3. Recommended browser: Google Chrome (version 90+) or Microsoft Edge. Firefox and Safari are also supported.

*Note: Do NOT open the file through a local development server — the NASA API uses Access-Control-Allow-Origin: * which works from file:// origins directly.*

1.2 Interface Overview

The interface is divided into two main areas:

- **: contains the star list, JSON upload zone, and live NASA search.**
- **: shows the light curve, model comparison table, CNN panel, and SHAP/Grad-CAM visualisations.**

1.3 Loading Results from Colab (JSON Import)

After running the Colab notebook (see Section 2), JSON result files are downloaded to your computer. To load them into the prototype:

4. Look for the "Drop JSON files here" zone in the left sidebar.
5. Drag one or more .json files from your Downloads folder and drop them onto this zone. Alternatively, click the zone to open a file picker.
6. Each loaded star appears in the star list on the left. Click a star name to display its results.
7. The main panel updates with the light curve, all model metrics, Grad-CAM overlay, and SHAP chart.

Note: If you see an error like "JSON parse failed", the notebook did not complete Cell 16 successfully. Re-run the notebook from Cell 16 and download the JSON again.

1.4 Live NASA Fetch (No Colab Required)

You can explore any star from the NASA archive without running the Colab notebook:

8. Find the search box at the top of the left sidebar labelled "KOI ID or TIC ID".
9. Type a Kepler KOI identifier in the format K00755.01 (capital K, five-digit zero-padded number, dot, two-digit planet number), OR a TESS TIC identifier as a plain number (e.g., 261136679).
10. Press Enter or click the Search button.
11. The prototype fetches orbital parameters and stellar metadata directly from the NASA Exoplanet Archive and displays a synthetic phase-folded light curve.

Note: Live-fetched stars show a simulated light curve. Model comparison metrics (ML accuracy, F1, MCC, etc.) are not available for live-fetched stars — only the stellar metadata and transit parameters are shown. To see model metrics, run the star through the Colab notebook.

1.5 Reading the Model Comparison Table

The model comparison table has 10 columns:

Column	Name	Meaning
#	Rank	Row number (1 = first ML model, last row = 1D CNN)
Model	Model name	Classifier name; CNN shown at the bottom under "Deep Learning" divider
Accuracy	Accuracy	Percentage of correctly classified samples in the held-out test set
Precision	Precision	Of all samples predicted as "planet", the fraction that actually are planets
Recall	Recall	Of all actual planets, the fraction that the model correctly identified
AUC-ROC	AUC	Area under the Receiver Operating Characteristic curve; 0.5 = random, 1.0 = perfect
MCC	MCC	Matthews Correlation Coefficient; ranges -1 to +1. Best single metric for imbalanced data
CV F1 $\pm \sigma$	Cross-val F1	5-fold cross-validation mean F1 $\pm$ standard deviation. More reliable than single-split F1 for small datasets
F1 Score	F1	Single-split F1 shown as a bar chart for quick visual comparison
This Star	Prediction	The model's prediction for the currently selected star: Planet or Non-Planet, with confidence %

Cells highlighted in green indicate the best value in that column across all models.

1.6 Light Curve and Grad-CAM

The light curve panel shows the phase-folded flux with phase on the x-axis (0 to 1) and normalised flux on the y-axis. When Grad-CAM data is available (from a loaded JSON), a colour overlay highlights the regions the CNN attended to most strongly. The transit window (phase 0.45–0.55) is marked with a dashed rectangle. A Grad-CAM localisation score is shown below the chart; values above 0.5 indicate that the CNN correctly focused on the transit region.

1.7 Opening Google Colab

A Colab badge ("Open Notebook") is displayed in the top-right corner of the prototype header. Clicking it opens the Google Colab website where you can upload and run the `exoplanet_colab.ipynb` notebook.

Section 2 — Colab Notebook Guide

The file `exoplanet_colab.ipynb` is a Google Colab notebook that trains all ML models and the 1D CNN on Kepler light curves and exports result JSON files for use in the prototype.

2.1 Setup

12. Go to <https://colab.research.google.com>
13. Click "Upload notebook" and select `exoplanet_colab.ipynb` from your project folder.
14. Change runtime: Runtime menu → Change runtime type → Hardware accelerator → T4 GPU (recommended for faster CNN training). Click Save.

2.2 Cell-by-Cell Reference

Cell	Name	What it does / What to configure
0	Intro	Markdown overview. Read for orientation. No code to run.
1	Install deps	Installs <code>lightcurve</code> , <code>xgboost</code> , <code>shap</code> , <code>lightgbm</code> . Run first; takes ~60 seconds.
2	Google Drive	Optional: set <code>USE_DRIVE = True</code> to save results to Google Drive. Leave <code>False</code> for local download.
3	Star config	EDIT HERE: add/remove <code>KOI_IDS</code> and <code>TIC_IDS</code> . Also set <code>SEQ_LEN</code> (default 1000), <code>CNN_EPOCHS</code> (40), <code>CNN_LR</code> ($5e-4$).
4	Download LC	Downloads light curves from NASA MAST using <code>lightcurve</code> . May take several minutes for 25 stars.
5	Preprocess	Applies Savitzky-Golay detrending, median normalisation, and resampling to <code>SEQ_LEN</code> points.
6	ML training	Trains 9 classifiers on 17 features. Prints a comparison table. Runs SHAP on XGBoost. ~2–5 min.
7	CNN training	Trains the 1D CNN. Progress bar shown per epoch. GPU runtime: ~3 min. CPU: ~15 min.
8	Grad-CAM + export	Computes Grad-CAM for each star, assembles the result JSON, and saves one <code>.json</code> per star.
9	Summary chart	Prints and plots a full model comparison bar chart.
10	Download JSON	Downloads all <code>.json</code> files to your computer. Each file is auto-named after the KOI/TIC ID.
11	Zip download	Optional: downloads all results (JSON + PNGs) as a single <code>.zip</code> archive.

2.3 Configuring the Star List (Cell 3)

The `KOI_IDS` list in Cell 3 is the primary configuration point. Each string in the list must be a valid KOI identifier from the NASA Exoplanet Archive cumulative table. The format is:

`K{5-digit-number}.{2-digit-planet-number}` e.g., `K00755.01`

Stars in the list with a **CONFIRMED** or **CANDIDATE** disposition receive label 1 (planet). Stars with **FALSE POSITIVE** disposition receive label 0. The disposition is read automatically from the NASA archive; you do not need to set labels manually.

For best model performance, include a balanced number of confirmed planets and false positives. The default configuration uses 15 confirmed and 10 false positives. To add more stars, append their KOI IDs to the list. To add TESS stars, append TIC IDs (integers) to the TIC_IDS list.

2.4 Troubleshooting

Problem	Solution
lightkurve returns no data for a star	The star may have no public SAP flux in MAST. Remove the KOI from the list or try a different quarter.
Only 1 class found — ValueError	All stars in your list have the same label. Add known false positive KOIs (e.g., K00005.01, K00012.01).
LightGBM not found	Cell 6 skips LightGBM if not installed. Re-run Cell 1 (installs lightgbm) then re-run Cell 6.
CNN loss is NaN	Reduce CNN_LR to 1e-4 or 5e-5 in Cell 3, then re-run Cells 7–8.
JSON import error in prototype	Cell 8 did not complete successfully. Check for Python errors in Cell 8, fix, and re-run. The <code>_san()</code> sanitiser should prevent NaN in JSON; if missing, ensure Cell 8 has the latest code.
Download not triggered	Pop-up blockers may suppress the auto-download. Click the file name link in the Colab output directly.

Section 3 — API and Data Sources

3.1 NASA Exoplanet Archive — nstedAPI

The prototype fetches data using the NASA nstedAPI (legacy CGI endpoint), which supports cross-origin requests from any origin including file:// URLs:

Base URL:
<https://exoplanetarchive.ipac.caltech.edu/cgi-bin/nstedAPI/nph-nstedAPI>

Request parameters are appended as URL query strings. Example request for a single KOI:

```
?table=cumulative&select=kepoi_name,koi_period,koi_depth,koi_duration,koi_pdisposition,koi_steff,koi_slogg,koi_smet,koi_srad,koi_smass&where=kepoi_name like 'K00755.01'&format=json
```

Important URL encoding rules:

- Spaces in where clauses encode as %20, which the NASA CGI correctly decodes to spaces.
- Do NOT use + as a space separator. The + character encodes to %2B, which the Oracle SQL backend treats as arithmetic concatenation, causing ORA-00936 errors.
- Single quotes in the where value must be included literally in the URL (not encoded).

3.2 KOI Cumulative Table Fields

Field Name	Type	Description
kepoi_name	string	KOI identifier, e.g. K00755.01 (key field for WHERE clause)
kepid	integer	Kepler Input Catalogue star identifier
koi_pdisposition	string	Disposition: CONFIRMED, FALSE POSITIVE, or CANDIDATE
koi_period	float	Orbital period in days
koi_duration	float	Transit duration in hours
koi_depth	float	Transit depth in parts per million (ppm)
koi_steff	float	Stellar effective temperature in Kelvin
koi_slogg	float	Stellar surface gravity (log g, cgs)
koi_smet	float	Stellar metallicity [Fe/H] relative to solar
koi_srad	float	Stellar radius in solar radii
koi_smass	float	Stellar mass in solar masses

3.3 TOI Table Fields (TESS Objects of Interest)

Field Name	Type	Description
tid	integer	TESS Input Catalogue (TIC) identifier (key field)

Field Name	Type	Description
tfopwg_disp	string	TFOPWG disposition: PC (planet candidate), KP (known planet), FP (false positive), etc.
pl_orbper	float	Orbital period in days
st_tmag	float	TESS magnitude of the host star

3.4 NASA MAST — Light Curve Download (Colab only)

Light curves are not fetched from the browser; they are downloaded inside the Colab notebook using the lightcurve Python library. Lightcurve queries the MAST archive via the astroquery interface:

```
lk.search_lightcurve(target="KIC {kepid}", mission="Kepler")
```

The SAP_FLUX (Simple Aperture Photometry) column is used. Lightcurve automatically stacks multiple quarters. The downloaded light curve is phase-folded using the orbital period and transit epoch from the KOI table, then preprocessed as described in Section 3 of the thesis report.

3.5 JSON Result File Format

Each star's Colab output is a JSON file with the following top-level structure:

```
{ "id": "K00755.01",
  "koi": { "period": 17.83, "depth": 4748.0, "duration": 3.61, ... },
  "prediction": "planet",
  "cnnConf": 0.87,
  "flux400": [ 0.9998, 1.0001, ... ],          // 400-point downsampled
flux
  "gradcam400": [ 0.01, 0.03, ... ],          // 400-point Grad-CAM
activation
  "models": {                                // 9 ML model metrics
    "Random Forest": { "accuracy": 0.8, "precision": 0.85, "recall":
0.75,
                        "f1": 0.80, "auc": 0.88, "mcc": 0.62,
                        "cv_f1_mean": 0.77, "cv_f1_std": 0.09,
                        "tp": 3, "fp": 1, "fn": 1, "tn": 4 }, ... },
    "cnn": { "accuracy": 0.85, "precision": 0.90, "recall": 0.80,
            "f1": 0.84, "auc": 0.91, "mcc": 0.70 },
    "model_preds": { "Random Forest": { "pred": "planet", "prob":
0.82 }, ... },
    "shap": [ { "feature": "transit_depth", "value": 0.32, "pos":
true }, ... ],
    "explainability": { "shap_faithfulness_drop": 0.15,
                        "shap_sparsity": 0.42,
                        "shap_consistency": 0.91,
                        "gradcam_localization": 0.63 }
  }
```

Section 4 — ML Models and Metrics Reference

4.1 Model Descriptions

Logistic Regression

A linear classifier that models the log-odds of the positive class as a linear combination of the feature vector. Despite its simplicity, it provides a useful baseline and is well-calibrated. Class imbalance is corrected by setting `class_weight="balanced"`. Regularisation parameter `C=1.0`.

Decision Tree

A non-parametric classifier that partitions the feature space using axis-aligned splits. Prone to overfitting on small datasets; `max_depth=6` and `min_samples_leaf=2` are set to limit complexity. `class_weight="balanced"` is applied.

k-Nearest Neighbours (KNN)

An instance-based classifier that predicts the majority class among the `k` nearest training samples in feature space. `k` is constrained to `min(3, n_train-1)` to avoid degenerate behaviour on small datasets. Does not support `class_weight`; imbalance is partially addressed by the feature scaling applied beforehand.

Support Vector Machine (SVM)

An SVM with RBF kernel, `C=2.0`, and `class_weight="balanced"`. The `probability=True` setting enables `predict_proba` via Platt scaling, required for AUC-ROC computation. Features must be scaled (StandardScaler applied).

Random Forest

An ensemble of 200 decision trees, each trained on a bootstrap sample of the training data with a random feature subset at each split. `max_depth=8` and `min_samples_leaf=2` limit individual tree complexity. `class_weight="balanced"` is applied. Random state is fixed (42) for reproducibility.

Extra Trees

Similar to Random Forest but uses fully random split thresholds (not optimised splits), resulting in greater variance reduction at the cost of slightly higher bias. Configuration mirrors Random Forest.

MLP (Multi-Layer Perceptron)

A feed-forward neural network with hidden layers of size (128, 64). `early_stopping=True` monitors validation loss and halts training if no improvement is seen, preventing overfitting. `max_iter=600` is the maximum epoch limit. Features must be scaled.

XGBoost

A gradient boosting framework using 300 trees with `learning_rate=0.03`, `max_depth=6`, `subsample=0.8`, `colsample_bytree=0.8`. `scale_pos_weight=n_negative/n_positive` directly corrects for class imbalance in the gradient computation. Regularisation: `reg_alpha=0.1` (L1), `reg_lambda=1.0` (L2). Trained with early stopping on the validation set.

LightGBM

A histogram-based gradient boosting implementation. Configuration mirrors XGBoost (300 trees, `lr=0.03`, `max_depth=6`) with `class_weight="balanced"`. Substantially faster than XGBoost on large datasets; comparable performance on small ones.

1D CNN

Three convolutional blocks (Conv → BN → ReLU → MaxPool → Dropout) followed by global average pooling and a two-class fully connected head. Trained with AdamW ($\text{lr}=5\text{e-}4$), cross-entropy loss with class weights, for 40 epochs with early stopping (patience 10).

4.2 Metrics Reference

Metric	Formula	Range	Interpretation
Accuracy	$(\text{TP}+\text{TN})/(\text{TP}+\text{TN}+\text{FP}+\text{FN})$	0 – 1 (higher = better)	Misleading under class imbalance; use as secondary metric
Precision	$\text{TP}/(\text{TP}+\text{FP})$	0 – 1 (higher = better)	Low precision = many false planet alerts
Recall	$\text{TP}/(\text{TP}+\text{FN})$	0 – 1 (higher = better)	Low recall = many real planets missed
F1 Score	$2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$	0 – 1 (higher = better)	Balances precision and recall; use for imbalanced data
AUC-ROC	Area under ROC curve	0.5 – 1.0 (higher = better)	0.5 = random, 1.0 = perfect; threshold-independent
MCC	$(\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}) / \text{sqrt}(\dots)$	-1 to +1 (higher = better)	Best single metric for imbalanced binary classification
CV F1 mean	Mean F1 over k folds	0 – 1 (higher = better)	More reliable than single-split F1 for small datasets
CV F1 std	Std dev over k folds	0 – 1 (lower = better)	High std = performance is split-dependent / unreliable

4.3 Understanding MCC

The Matthews Correlation Coefficient is computed as:

$$\text{MCC} = (\text{TP} \times \text{TN} - \text{FP} \times \text{FN}) / \text{sqrt}((\text{TP}+\text{FP})(\text{TP}+\text{FN})(\text{TN}+\text{FP})(\text{TN}+\text{FN}))$$

It considers all four quadrants of the confusion matrix equally. A model that always predicts the majority class achieves $\text{MCC} \approx 0$ regardless of accuracy, correctly reflecting its worthlessness as a classifier. A perfect classifier achieves $\text{MCC} = +1$. Negative MCC values indicate a classifier that systematically inverts the correct label, which is worse than random and usually indicates a labelling error or severe feature mismatch.

For the exoplanet dataset with approximately 15 planets and 10 false positives, a model achieving accuracy of 0.75 by predicting "planet" for everything would show $\text{MCC} \approx 0$. By contrast, a genuinely discriminative model achieving 0.75 balanced accuracy would show $\text{MCC} > 0.4$.

4.4 Understanding Cross-Validation F1

With only 25 training samples, a single 80/20 train/test split places only 5 samples in the test set. Performance on 5 samples is highly variable; one misclassification changes F1 by 0.2 or more. Stratified 5-fold cross-validation uses each sample in exactly one test fold across the five iterations, producing five F1 measurements from which mean $\pm$ standard deviation are computed. This provides:

- A more stable estimate of expected performance on unseen data.
- A confidence interval via the standard deviation: $cv_f1_mean \pm cv_f1_std$.
- Detection of overfit models: a model with high single-split F1 but high cv_f1_std is likely overfit to the particular train/test split.

4.5 SHAP Explainability Metrics

Metric	Formula	Interpretation
Faithfulness drop	$Acc(full) - Acc(top-3 \text{ masked})$	Higher = explanation identifies genuinely impactful features
Sparsity	Fraction of $ SHAP < 0.01$	Higher = model relies on fewer features (more interpretable)
Consistency	Mean Spearman rank corr. across samples	Near 1.0 = stable, reproducible explanation across inputs